



UNIVERSIDADE PAULISTA – UNIP
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS

EDUARDO LOURENÇO SILVA
DANIEL VALÉRIO DOS SANTOS
KEVIN CLAYTON ESTACA ORELLANA
PEDRO HENRIQUE FERREIRA SANTOS
RHUAN CAVALCANTE SILVA

TECHPAPER

Continuidade do sistema: aplicação móvel em JavaScript e banco de dados PostgreSQL

SÃO PAULO
2026

EDUARDO LOURENÇO SILVA
DANIEL VALÉRIO DOS SANTOS
KEVIN CLAYTON ESTACA ORELLANA
PEDRO HENRIQUE FERREIRA SANTOS
RHUAN CAVALCANTE SILVA

TECHPAPER

Continuidade do sistema: aplicação móvel em JavaScript e banco de dados PostgreSQL

Trabalho apresentado no Projeto Integrado Multidisciplinar IV do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Paulista – UNIP, como continuidade do sistema desenvolvido no PIM III.

Orientadora: Profa. Priscila Faccoli.

SÃO PAULO

2026

RESUMO

Este trabalho apresenta a continuidade do sistema TechPaper, criado para apoiar o gerenciamento de papelarias. A etapa desenvolvida para o Projeto Integrado Multidisciplinar IV acrescenta uma aplicação móvel em JavaScript, integra as interfaces web e móvel à mesma API REST e amplia a programação do banco de dados PostgreSQL. O aplicativo Android foi implementado com React Native e Expo e possui autenticação, visão resumida, pesquisa de produtos, movimentação de estoque, elaboração de orçamentos e gerenciamento da sessão. A sincronização consulta a API por HTTPS e utiliza armazenamento protegido para a sessão. No banco, foram estruturadas oito tabelas, restrições, chaves estrangeiras, índices, funções em PL/pgSQL e gatilhos de auditoria. A movimentação de estoque utiliza bloqueio transacional e uma chave de operação para impedir saldo negativo e lançamentos duplicados. Testes automatizados verificam autorização, integridade, concorrência, idempotência, cálculo de orçamentos e comunicação do aplicativo. O resultado mantém a identidade funcional do PIM III e acrescenta os elementos solicitados pelo manual do PIM IV para desenvolvimento mobile, JavaScript e banco de dados.

Palavras-chave: aplicação móvel; JavaScript; React Native; PostgreSQL; API REST.

ABSTRACT

This paper presents the continuation of TechPaper, a system designed to support stationery store management. The PIM IV stage adds a JavaScript mobile application, connects the web and mobile interfaces to the same REST API, and expands PostgreSQL database programming. The Android application uses React Native and Expo and provides authentication, overview indicators, product search, inventory transactions, quotations, and session management. Synchronization uses HTTPS and protected session storage. The database contains eight tables, constraints, foreign keys, indexes, PL/pgSQL functions, and audit triggers. Inventory transactions apply row locking and an operation key to prevent negative balances and duplicate records. Automated tests cover authorization, integrity, concurrency, idempotency, quotation totals, and application communication. The result preserves the functional identity of PIM III and adds the elements required by the PIM IV manual for mobile development, JavaScript, and database programming.

Keywords: mobile application; JavaScript; React Native; PostgreSQL; REST API.

SUMÁRIO

1 INTRODUÇÃO	6
2 CONTINUIDADE E ARQUITETURA DO TECHPAPER	7
3 DESENVOLVIMENTO DA SOLUÇÃO MOBILE	8
4 DESENVOLVIMENTO MOBILE COM JAVASCRIPT	10
5 PROJETO E PROGRAMAÇÃO DO BANCO DE DADOS	11
6 INTEGRAÇÃO ENTRE MOBILE, API E BANCO	13
7 TESTES E RESULTADOS	14
8 DETALHAMENTO DO MODELO FÍSICO	16
9 CONSIDERAÇÕES FINAIS	17
REFERÊNCIAS	18
APÊNDICE A – ROTEIRO DE EVIDÊNCIAS	19
APÊNDICE B – CONFORMIDADE COM O MANUAL	20
APÊNDICE C – COMO INCORPORAR AO PIM ORIGINAL	21

1 INTRODUÇÃO

O TechPaper foi concebido no PIM III como um sistema web de orçamento e gerenciamento de papelarias. A continuidade proposta pelo PIM IV conserva o cadastro de usuários, fornecedores e produtos, o controle de estoque, a elaboração de orçamentos e a consulta de indicadores. A ampliação concentra-se em três frentes relacionadas às disciplinas do semestre: desenvolvimento para dispositivos móveis, desenvolvimento mobile com JavaScript e projeto e programação de banco de dados.

O objetivo geral é disponibilizar uma solução integrada em que o portal e o aplicativo Android consultem a mesma fonte de dados e obedeçam às mesmas regras de negócio. Os objetivos específicos são construir as telas principais do aplicativo, definir um fluxo de navegação simples, autenticar os funcionários, sincronizar dados com uma API REST, proteger a sessão, evitar a duplicidade de movimentações, modelar o banco relacional, implementar rotinas e gatilhos e produzir evidências de funcionamento.

A metodologia combinou análise dos requisitos do sistema anterior, desenho da arquitetura, implementação incremental e testes automatizados. A camada de serviços foi centralizada na API ASP.NET Core, o aplicativo foi desenvolvido em React Native com Expo e a persistência foi migrada para PostgreSQL no serviço Neon. A API e o portal são publicados no Render.

O Manual do PIM IV exige telas principais, fluxo de navegação, autenticação, sincronização e integração com APIs. Para banco de dados, solicita modelo lógico, modelo físico, modelo entidade relacionamento, procedimentos, gatilhos e script completo. Os capítulos seguintes respondem diretamente a esses itens.

2 CONTINUIDADE E ARQUITETURA DO TECHPAPER

Portal web e aplicativo móvel atuam como clientes da mesma API REST. A API autentica cada requisição, verifica o perfil, valida os dados, executa as regras e persiste as alterações. Essa organização impede que uma tela altere preços, totais e saldos sem passar pelas regras centrais.

Arquitetura de integração do TechPaper

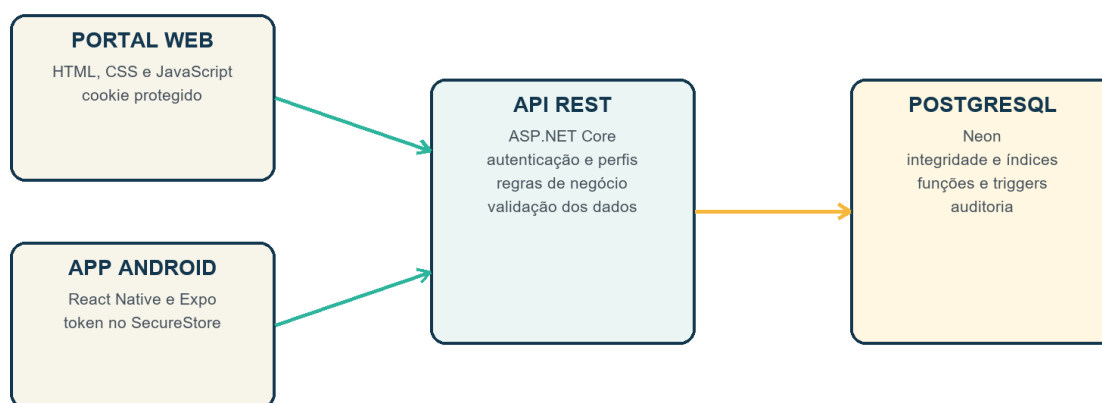


Figura 1 – Arquitetura de integração do TechPaper

Fonte: autoria própria (2026).

A comunicação pública utiliza HTTPS. O portal recebe um cookie de sessão restrito ao servidor, enquanto o aplicativo recebe um token e o armazena no SecureStore. O banco mantém apenas o hash do token. O logout solicita a revogação da sessão e exclui a cópia local.

Camada	Tecnologia	Responsabilidade
Portal	HTML, CSS e JavaScript	Interface administrativa e visão geral
Aplicativo	React Native, Expo e JavaScript	Operação móvel e sincronização
Serviço	ASP.NET Core e API REST	Autenticação, autorização e regras
Persistência	PostgreSQL no Neon	Integridade, concorrência e auditoria
Hospedagem	Render e EAS	Publicação do serviço e Android

Fonte: autoria própria (2026).

3 DESENVOLVIMENTO DA SOLUÇÃO MOBILE

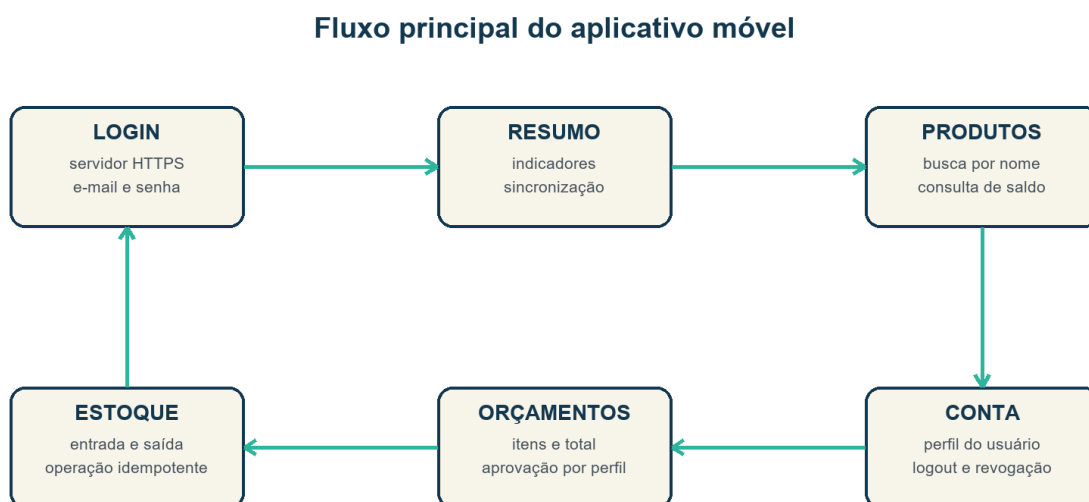


Figura 2 – Fluxo principal do aplicativo móvel

Fonte: autoria própria (2026).

Como continuidade do sistema TechPaper apresentado no PIM III, foi desenvolvida uma aplicação móvel destinada aos funcionários da papelaria. O aplicativo utiliza a mesma API REST consumida pelo portal web, permitindo que produtos, movimentações de estoque e orçamentos sejam consultados a partir de fontes de dados comuns. Essa integração evita a manutenção de cadastros independentes e reduz o risco de divergência entre as aplicações.

A solução foi implementada em JavaScript com React Native e Expo. O manual do PIM IV não determina uma tecnologia específica para a aplicação móvel, mas exige a apresentação das telas principais, fluxo de navegação, autenticação, sincronização e integração com APIs. A escolha adotada permitiu atender a esses itens e aplicar os conhecimentos da disciplina de desenvolvimento mobile com JavaScript. O React Native fornece componentes que são convertidos em interfaces nativas, enquanto o Expo organiza o ambiente de desenvolvimento e os módulos utilizados pelo aplicativo.

As telas principais são Login, Resumo, Produtos, Estoque, Orçamentos e Conta. Após a autenticação, o usuário acessa um resumo dos produtos cadastrados, itens com estoque baixo e orçamentos em rascunho. A tela Produtos permite pesquisar pelo nome ou pelo código SKU. Em Estoque, o funcionário seleciona um produto, informa se a movimentação é uma entrada ou saída, registra a quantidade e descreve o motivo. O módulo Orçamentos permite selecionar produtos e quantidades, salvar a proposta e acompanhar seu estado. A tela Conta identifica o usuário e seu perfil, apresenta orientações de inclusão e encerra a sessão.

O fluxo de navegação utiliza abas inferiores para acesso direto às funções principais. O botão de retorno do sistema fecha o detalhe do orçamento ou retorna ao Resumo antes de sair do aplicativo. Botões, campos e itens selecionáveis possuem papéis e estados de acessibilidade, e os campos de formulário incluem rótulos textuais. Essas decisões facilitam o uso com o TalkBack e preservam a operação com tamanhos de fonte definidos no aparelho. A documentação do React Native descreve propriedades que permitem integrar os componentes às tecnologias assistivas do Android e do iOS (REACT NATIVE, 2026).

4 DESENVOLVIMENTO MOBILE COM JAVASCRIPT

A camada de comunicação foi centralizada no módulo `src/api.mjs`. Esse módulo recebe o endereço do servidor, acrescenta os cabeçalhos exigidos pela API, serializa os dados em JSON e limita o tempo de espera das requisições. Respostas de erro são convertidas em mensagens compreensíveis e mantêm o código HTTP, permitindo que a interface reconheça, por exemplo, uma sessão expirada. Em uma implantação, o aplicativo aceita apenas endereços HTTPS. Endereços HTTP são permitidos somente no modo de desenvolvimento para testes na rede local.

Após o login, o token da sessão é armazenado pelo Expo SecureStore. Esse módulo utiliza mecanismos de armazenamento protegido oferecidos pelo sistema operacional, evitando o uso de armazenamento comum para a credencial da sessão (EXPO, 2026). A senha permanece apenas no estado da tela durante o login e é removida após a autenticação. Ao sair, o aplicativo solicita a revogação da sessão na API e apaga o token local.

A sincronização é realizada no login, pelo gesto de puxar a tela, pelo botão Atualizar e quando o aplicativo retorna ao primeiro plano. Produtos, movimentações e orçamentos são consultados em paralelo. A interface só substitui os dados apresentados quando todas as consultas são confirmadas. Se a sessão expirar, o token é apagado e o usuário retorna à tela de acesso.

As movimentações de estoque exigem cuidado adicional porque a perda da resposta não informa se o servidor concluiu a operação. Antes de enviar a requisição, o aplicativo gera uma identificação única e armazena a tentativa. Caso não receba confirmação, a próxima tentativa reutiliza essa identificação. A função do banco reconhece a repetição e devolve a movimentação já criada sem alterar novamente o estoque. Dessa forma, uma instabilidade de rede não transforma uma entrada ou saída em duas operações.

O cálculo financeiro definitivo não ocorre no JavaScript. Ao criar um orçamento, o aplicativo envia apenas o cliente, a validade, o produto e a quantidade. A API consulta o preço cadastrado, calcula subtotais e total e devolve o orçamento confirmado. Isso evita que uma alteração local no aplicativo determine o valor gravado. O orçamento, por decisão inicial do projeto, não reserva nem reduz estoque; uma saída é registrada quando a venda ou entrega é confirmada. A equipe deve validar essa regra antes da apresentação.

5 PROJETO E PROGRAMAÇÃO DO BANCO DE DADOS

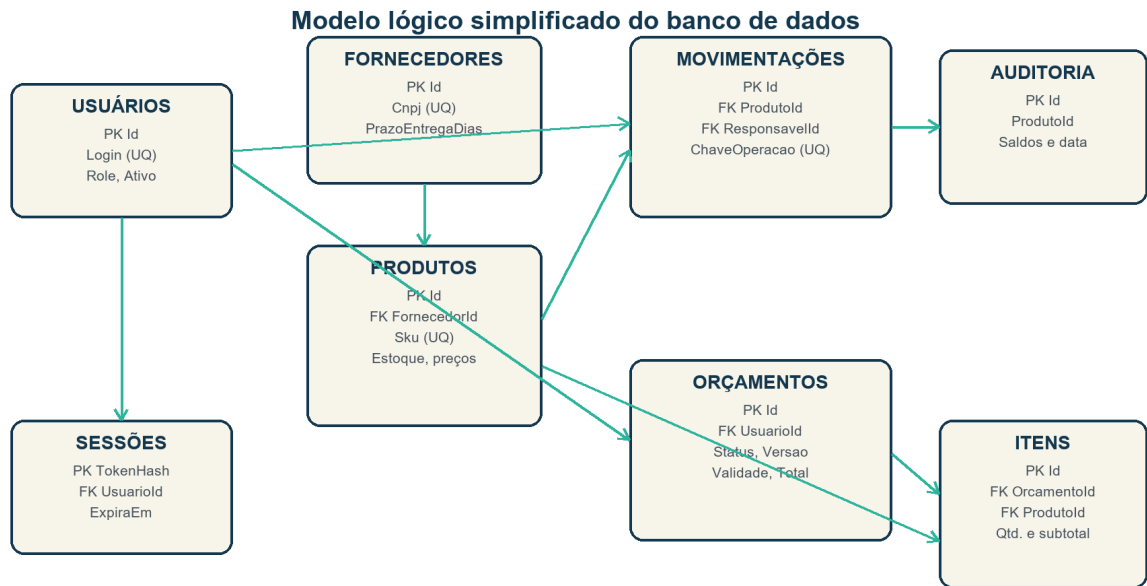


Figura 3 – Modelo lógico simplificado do banco de dados

Fonte: autoria própria (2026).

O banco do PIM IV foi ampliado para representar usuários, fornecedores, produtos, movimentações, sessões, orçamentos, itens de orçamento e auditoria de estoque. O PostgreSQL foi escolhido porque oferece transações, integridade referencial, índices, funções armazenadas e gatilhos e pode ser utilizado na camada gratuita da Neon sem prazo de 30 dias. O script completo está dividido em arquivos numerados e a API pode aplicá-lo automaticamente em um banco vazio.

As tabelas utilizam chaves primárias numéricas e chaves estrangeiras para representar os vínculos. Cada produto referencia um fornecedor; cada movimentação referencia o produto e o usuário responsável; cada orçamento pertence ao usuário que o criou; e os itens relacionam o orçamento aos produtos. Restrições impedem preços, quantidades e saldos negativos. Índices foram criados para SKU, nome e categoria do produto, produtos com estoque baixo, histórico por produto e data, sessões expiradas e orçamentos por estado e data.

A função PL/pgSQL `sp_registrar_movimentacao` concentra a alteração do estoque. Ela valida tipo e quantidade, bloqueia o produto selecionado com `FOR UPDATE`, verifica o saldo e atualiza o produto antes de inserir o histórico. Em uma saída, a operação é rejeitada se a quantidade solicitada ultrapassar o saldo. Como a função participa da mesma transação iniciada pela chamada da API, duas saídas simultâneas não podem utilizar o mesmo saldo disponível. O PostgreSQL permite implementar funções e gatilhos com estruturas de

controle e comandos SQL no servidor (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026).

Foram criados três triggers. O primeiro registra o saldo anterior e o saldo atual em `auditoria_estoque` após cada mudança de estoque. Os outros dois impedem a edição e a exclusão das movimentações. Quando um lançamento precisa ser corrigido, o usuário deve registrar uma movimentação compensatória. Essa regra mantém a rastreabilidade das operações e permite comparar o saldo atual com o histórico.

A função `sp_resumo_estoque` recebe um limite e apresenta os produtos abaixo desse valor, com custo unitário e valor armazenado. O arquivo de verificação também compara o saldo de cada produto à soma das entradas e saídas, compara o total do orçamento à soma dos itens e utiliza EXPLAIN para demonstrar o índice da consulta de histórico.

6 INTEGRAÇÃO ENTRE MOBILE, API E BANCO

A API foi organizada em modelos, contratos, controladores, serviços, segurança e persistência. Os contratos limitam tamanho, formato e faixa dos dados recebidos. Os controladores definem as rotas e permissões. Os serviços executam as regras de estoque e orçamento. O Entity Framework Core com Npgsql relaciona os objetos às tabelas PostgreSQL. Essa separação reduz a quantidade de regras nas interfaces e permite que web e mobile compartilhem o mesmo comportamento.

A autenticação determina a identidade do usuário e a autorização define as operações permitidas por seu perfil, conceitos distintos no ASP.NET Core (MICROSOFT, 2026). O operador consulta dados, movimenta estoque e cria orçamentos. O supervisor também mantém produtos e fornecedores e altera o estado de orçamentos. O administrador também gerencia usuários. A API aplica essas regras mesmo quando uma requisição é realizada fora da interface.

As senhas são transformadas por um algoritmo de hash antes do armazenamento e nunca aparecem nas respostas. O token recebido pelo aplicativo também não é gravado diretamente no banco: armazena-se apenas seu hash. Sessões podem ser revogadas no logout, na alteração administrativa do usuário ou quando chegam ao prazo de expiração. O portal utiliza cookie restrito ao servidor e o aplicativo envia o token no cabeçalho de autorização.

7 TESTES E RESULTADOS

A API foi compilada com o SDK .NET 10.0.400 sem erros ou avisos. Em um banco descartável foram executados nove testes de integração. Eles verificaram bloqueio de acesso sem sessão, restrição administrativa, ausência de senhas nas respostas, identidade da movimentação, repetição segura, duas retiradas simultâneas, saldo insuficiente, validação de quantidades, cálculo do orçamento, restrição de aprovação, conflito de versões, rejeição de origem externa e revogação da sessão.

No cenário de concorrência, o produto possuía dez unidades e duas requisições tentaram retirar sete unidades ao mesmo tempo. Uma foi confirmada e a outra recebeu conflito, preservando saldo final igual a três. No cenário de repetição, duas chamadas com a mesma identificação devolveram o mesmo registro e alteraram o estoque uma única vez. Também foram executados seis testes do módulo de comunicação JavaScript, cobrindo endereço seguro, quantidades, agrupamento de itens, envio do token, expiração da sessão e falha de rede.

O portal foi verificado no navegador com a criação de um orçamento contendo duas unidades de um produto de R\$ 24,90. A API gravou o total de R\$ 49,80 e o orçamento apareceu na listagem em estado de rascunho. A equipe ainda deve executar o aplicativo em aparelho Android, capturar as telas e repetir o teste de sincronização na rede que será utilizada durante a apresentação.

8 REFERÊNCIAS PARA ESTA SEÇÃO

EXPO. SecureStore. Documentação do Expo. Disponível em: <https://docs.expo.dev/versions/latest/sdk/securestore/>. Acesso em: 6 set. 2026.

MICROSOFT. Overview of ASP.NET Core authentication. Microsoft Learn. Disponível em: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/?view=aspnetcore-10.0>. Acesso em: 6 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PL/pgSQL — SQL Procedural Language. PostgreSQL 18 Documentation. 2026. Disponível em: <https://www.postgresql.org/docs/current/plpgsql-overview.html>. Acesso em: 6 set. 2026.

NEON. Neon pricing. 2026. Disponível em: <https://neon.com/pricing>. Acesso em: 6 set. 2026.

REACT NATIVE. Accessibility. Documentação do React Native. Disponível em: <https://reactnative.dev/docs/accessibility.html>. Acesso em: 6 set. 2026.

8 DETALHAMENTO DO MODELO FÍSICO

O PIM III descreveu uma persistência inicial em MySQL. Na continuidade, o modelo foi revisado e implementado em PostgreSQL. Nos capítulos atuais, as referências a MySQL devem ser tratadas como histórico da primeira versão. A versão executada pela API, pelo portal e pelo aplicativo utiliza PostgreSQL no Neon.

Tabela	Chave	Campos e finalidade
usuarios	Id	Name, Login, Password, Role, Ativo e DataCriacao
fornecedores	Id	Cnpj, RazaoSocial, NomeFantasia, contato e prazo
produtos	Id	Sku, Nome, Categoria, FornecedorId, preços e Estoque
movimentacoes	Id	Tipo, ProdutoId, ResponsavelId, quantidade, chave e DataHora
sessoes	TokenHash	UsuarioId e ExpiraEm
orcamentos	Id	Cliente, Status, UsuarioId, Validade, Total e Versao
itens_orcamento	Id	OrcamentoId, ProdutoId, Quantidade, preço e Subtotal
auditoria_estoque	Id	ProdutoId, saldos anterior e atual e DataHora

Fonte: autoria própria (2026).

Chaves primárias, chaves estrangeiras, unicidade e restrições CHECK preservam a integridade. Login, CNPJ, SKU e chave de operação são únicos. Índices apoiam a busca de produtos, o baixo estoque, o histórico por produto e data, as sessões vencidas e os orçamentos por estado.

O arquivo database/000_banco_completo.sql reúne toda a criação do banco dentro de uma transação e pode ser executado no SQL Editor do Neon. O arquivo database/004_verificacao.sql lista tabelas, funções e gatilhos, compara saldos e totais e apresenta o plano da consulta de histórico.

9 CONSIDERAÇÕES FINAIS

A continuidade do TechPaper integrou as três disciplinas ao sistema do semestre anterior. O desenvolvimento mobile aplicou telas, navegação, autenticação, sincronização e acessibilidade. O JavaScript estruturou requisições assíncronas, estado, ciclo de vida, validação, tratamento de falhas e idempotência. O banco definiu o modelo relacional e físico, funções, gatilhos, índices, transações e auditoria.

A centralização das regras na API e no banco evita divergências entre portal e aplicativo. O bloqueio transacional protege o saldo, a chave de operação protege contra repetição e os gatilhos preservam a rastreabilidade. A solução pode evoluir com notificações de baixo estoque, leitura de código de barras e relatórios históricos, mantendo a arquitetura implementada.

REFERÊNCIAS

EXPO. Build an app with EAS Build. Disponível em: <https://docs.expo.dev/build/setup/>. Acesso em: 7 set. 2026.

EXPO. SecureStore. Disponível em: <https://docs.expo.dev/versions/latest/sdk/securestore/>. Acesso em: 7 set. 2026.

MICROSOFT. Overview of ASP.NET Core authentication. Microsoft Learn. Disponível em: <https://learn.microsoft.com/aspnet/core/security/authentication/>. Acesso em: 7 set. 2026.

NEON. Neon documentation. Disponível em: <https://neon.com/docs>. Acesso em: 7 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation: PL/pgSQL. Disponível em: <https://www.postgresql.org/docs/current/plpgsql.html>. Acesso em: 7 set. 2026.

REACT NATIVE. Accessibility. Disponível em: <https://reactnative.dev/docs/accessibility>. Acesso em: 7 set. 2026.

REACT NATIVE. Networking. Disponível em: <https://reactnative.dev/docs/network>. Acesso em: 7 set. 2026.

UNIVERSIDADE PAULISTA. Manual do Projeto Integrado Multidisciplinar IV: Análise e Desenvolvimento de Sistemas. Versão 2. São Paulo: UNIP, 2026.

APÊNDICE A – ROTEIRO DE EVIDÊNCIAS

Faça as capturas depois de instalar o APK. Não inclua senhas, tokens nem a conexão completa do banco nas imagens.

Ordem	Captura	Ação
1	Login do Android	Abra o TechPaper antes de entrar.
2	Resumo	Entre e aguarde a atualização.
3	Produtos	Pesquise pelo nome ou SKU.
4	Estoque	Registre entrada com motivo Demonstração PIM IV.
5	Sincronização	Atualize o portal e mostre a mesma movimentação.
6	Orçamento	Crie uma proposta e abra os detalhes.
7	Conta	Mostre perfil, inclusão e logout.
8	Visão geral web	Mostre indicadores e gráficos.
9	Neon	Mostre oito tabelas, funções e triggers.
10	Testes	Mostre o resultado final dos testes.

Fonte: autoria própria (2026).

APÊNDICE B – CONFORMIDADE COM O MANUAL

Exigência	Evidência
Telas principais	Login, Resumo, Produtos, Estoque, Orçamentos e Conta
Navegação	Figura 2 e abas inferiores
Autenticação	API, SecureStore e revogação
Sincronização	Login, gesto, botão e retorno ao primeiro plano
APIs	API REST comum ao portal e ao aplicativo
Modelo lógico e MER	Figura 3 e dicionário
Modelo físico	001_schema.sql e 000_banco_completo.sql
Procedimentos	sp_registrar_movimentacao e sp_resumo_estoque
Triggers	Auditoria e imutabilidade
Script completo	database/000_banco_completo.sql

Fonte: autoria própria (2026).

APÊNDICE C – COMO INCORPORAR AO PIM ORIGINAL

Mantenha no documento principal os elementos pré-textuais e a descrição organizacional já revisados. Insira os capítulos mobile após a apresentação do sistema web. Substitua capítulos atuais que afirmem que a versão final utiliza MySQL pelos capítulos de banco deste arquivo. Use os estilos Título 1, Título 2 e Título 3 para ajustar a numeração e atualize o sumário. Acrescente as capturas do Apêndice A e confira se o desenvolvimento completo do PIM permanece entre vinte e trinta páginas, conforme o manual.

Antes da entrega, o grupo deve confirmar nomes, orientadora, local e ano, revisar o texto, atualizar todos os campos do Word e gerar o PDF. A equipe precisa saber explicar o fluxo da movimentação, a diferença entre autenticação e autorização, o armazenamento protegido, a função transacional e os gatilhos.